

Compute4Me Study Notes — Section 4: Elastic and Heterogeneous Distributed Training

Context: Section 4 of the original report focused on *elastic* and *heterogeneous* distributed training. This document expands that section into a practical, implementation-oriented guide: what elastic training actually means, how systems like EasyScale, Horovod Elastic, PyTorch torchrun, and large-scale scheduling frameworks work, their difficulty to use vs reimplement, and what they imply for a Compute4Me-style system.

4.1 What Is “Elastic Training” in Practice?

Standard distributed data-parallel training assumes a fixed world size:

- You start with N GPUs.
- You finish with the same N GPUs.
- Batch size, learning rate, and communication patterns are all tuned to that N .

In real clusters (and especially in a Compute4Me-like volunteer fabric), this assumption breaks:

- New GPUs become free and can be added.
- Some nodes go down, preempt, or lose network.
- You may want to opportunistically borrow idle GPUs for a while.

Elastic training is about allowing the number of workers/GPUs to change during training, without throwing away progress or sacrificing convergence.

Concretely, an elastic system needs to handle:

1. Membership changes:

- Worker joins: training continues with $N + 1$ workers.
- Worker leaves/fails: training continues with $N - 1$ workers.

2. State continuity:

- Model weights, optimizer state, and training progress (epoch, iteration) must remain logically consistent.
- You want to avoid restarting from scratch whenever world size changes.

3. Convergence/accuracy consistency:

- Naively changing world size changes effective batch size → breaks your LR schedule.
- Elastic systems must adjust hyperparameters or internal algorithms so that final accuracy is comparable to non-elastic baselines.

For Compute4Me, elasticity is almost mandatory: you want to exploit whatever GPUs are online right now in a cluster of heterogeneous, possibly unreliable volunteer machines.

4.2 EasyScale: Accuracy-Consistent Elastic Training

4.2.1 What EasyScale Is

EasyScale is a system designed specifically to make data-parallel training elastic while preserving accuracy, even on a heterogeneous GPU cluster.

- Paper: "*EasyScale: Accuracy-consistent Elastic Training for Deep Learning*" (Liet al., 2022)
arXiv: <https://arxiv.org/abs/2208.14228>
- Talk (SC'23): "EasyScale: Elastic Training with Consistent Accuracy and Improved Utilization on GPUs"
Example: <https://www.youtube.com/watch?v=a63MWTkcSbM>

Problem it attacks:

- Standard synchronous data-parallel training needs a fixed number of GPUs.
- Changing GPU count mid-run typically changes effective batch size, messes up optimizer dynamics, and thus hurts accuracy.

EasyScale's goals:

- Allow GPU count to grow/shrink during training (elasticity).
- Maintain numerically equivalent behavior to a fixed-world training run — i.e., same update sequence as if you had trained with a fixed set of workers.
- Exploit idle GPUs in a production cluster to improve utilization.

4.2.2 Key Ideas (without implementation detail overload)

From the paper and talk, the central tricks are:

1. Strict preservation of data-parallel semantics:

- EasyScale wants training under elasticity to behave *as if* it were a fixed-number-of-GPUs job.
- That means the sequence of effective updates (gradients applied) should be equivalent.

2. Decouple training logic from resource allocation:

- They introduce abstractions so that the *training algorithm* does not depend directly on the current world size.
- Instead, the system carefully tracks "consistency-relevant factors" like global batch size and learning-rate scheduling, and compensates for changes.

3. EasyScaleThread abstraction:

- Conceptually, this is a software abstraction that represents a "logical worker thread" which can be mapped to different physical GPUs over time.
- It supports fast context switching when GPUs are added/removed.

4. Intra-job and inter-job schedulers:

- **Intra-job:** decides which GPUs are assigned to which training job as they become available or are reclaimed.
- **Inter-job:** coordinates across multiple jobs to minimize load imbalance and maximize global throughput.

Results:

- Deployed in an online serving cluster, EasyScale harnesses idle GPUs opportunistically and improves overall cluster utilization by ~62% while maintaining baseline accuracy.

4.2.3 Difficulty to Use vs Reimplement

- **Using EasyScale:**
 - EasyScale is a research system, not a widely packaged library like PyTorch DDP.
 - There is no public, plug-and-play PyPI package; you would likely need to adapt the ideas manually or study their code if available.
- **Reimplementing EasyScale concepts in your stack:**
 - **Medium-hard** if you already use PyTorch DDP or Horovod:
 - You can adopt some ideas, such as maintaining a constant effective global batch size and adjusting LR when world size changes.
 - You can use PyTorch's elastic `torchrun` (Section 4.3) as the low-level elastic engine, then add your own logic on top.
 - **Full fidelity (as in the paper): Hard** — they put significant effort into tracing all consistency factors and implementing fast context switching.

Relevance to Compute4Me: EasyScale is a blueprint for how to exploit changing GPU availability *without* wrecking convergence. For a volunteer-style system, the high-level lesson is more important than the exact implementation:

- Treat training as logically independent from the current GPU set.
- Track global batch size and LR carefully.
- Adjust your scheduler so new GPUs can be absorbed mid-run without retraining from scratch.

4.3 Elastic Training in Today's Framework Ecosystem

Elastic ideas have also been integrated (partially) into mainstream tooling:

- Horovod Elastic
- Horovod on Ray (ElasticRayExecutor)
- PyTorch torchrun elastic mode

4.3.1 Horovod Elastic

Horovod started as a fixed-world all-reduce library, but later added **elastic training** support.

- Horovod homepage: <https://horovod.ai>
- Horovod GitHub: <https://github.com/horovod/horovod>
- Horovod elastic docs: <https://horovod.readthedocs.io> (see elastic training sections)

Key concepts:

- You wrap your training loop with `@hvd.elastic.run`.
- There's a `TorchState` (or equivalent) object that tracks model, optimizer, epoch, and batch.
- On a membership change (node added or removed), Horovod:
 - Resets the world size
 - Adjusts LR / batch slicing as needed
 - Restarts workers from a consistent state

Example from docs (simplified):

```
import horovod.torch as hvd

hvd.init()
model = MyModel()
optimizer = optim.SGD(model.parameters(), lr=0.01)

@hvd.elastic.run
def train(state):
    for state.epoch in range(state.epoch, max_epochs):
        for state.batch, batch_data in enumerate(dataloader, start=state.batch):
            loss = model(batch_data).loss
            loss.backward()
            optimizer.step()
            state.commit() # mark state as consistent

state = hvd.elastic.TorchState(model, optimizer, batch=0, epoch=0)
train(state)
```

What Horovod Elastic gives you:

- If a worker dies or a new worker is added (supported cluster-side), training can **restart** from the last consistent state.

- Elasticity is tied to checkpointing via the state object.

Difficulty:

- Using it: moderate — you must structure your training loop around the Horovod elastic API and ensure state is checkpointed correctly.
- Reimplementing the mechanism from scratch: medium–high — involves state management, recovery protocol, and cluster coordination.

4.3.2 Horovod on Ray (ElasticRayExecutor)

Horovod integrates with Ray through a component called `RayExecutor`.^[web:112]

- Docs: https://horovod.readthedocs.io/en/latest/ray_include.html

Key points:

- `RayExecutor` launches Horovod workers as Ray actors on a Ray cluster.
- `ElasticRayExecutor` supports elastic Horovod training so that the number of Horovod workers can change at runtime.
- Ray's autoscaler can spin up or tear down cloud instances dynamically (e.g., AWS EC2), and Horovod Elastic adapts to these changes.

Example (simplified):

```
import ray
from horovod.ray import RayExecutor

ray.init(address="auto")
settings = RayExecutor.create_settings(verbose=True)
executor = RayExecutor(settings, min_workers=1, use_gpu=True)

executor.start()
executor.run(training_fn) # training_fn uses Horovod Elastic inside
```

Relevance to Compute4Me: This is very close to what Compute4Me wants for elastic training on top of a dynamic pool of nodes — but it assumes:

- A controlled Ray cluster with cloud autoscaling.
- Trusted nodes.

Compute4Me can borrow the **pattern** (a higher-level orchestrator that spins up/down workers) but adapt the **control plane** to volunteer Docker nodes instead of cloud VMs.

4.3.3 PyTorch `torchrun` Elastic Mode

PyTorch provides `torchrun` (formerly `torch.distributed.launch`) with support for fault-tolerant and elastic training.

- Tutorial: "Fault-tolerant Distributed Training with `torchrun`"
https://pytorch.org/tutorials/beginner/ddp_series_fault_tolerance.html

Key features:

- You write your training script **once** with a normal `main()`.
- `torchrun` handles launching multiple processes and environment setup.
- You structure your script around **snapshots**:
 - Frequently save state (model, optimizer, epoch, etc.).
 - On failure or membership change, `torchrun` restarts processes; they load the last snapshot and continue.

Elastic behavior:

- When a node joins or leaves, `torchrun` terminates processes and **respawns** them with updated `WORLD_SIZE` and `RANK` assignments.
- Your code just needs to load snapshot → initialize → continue training.

Example structure:

```
from torch.distributed.elastic.multiprocessing.errors import record

@record
def main():
    snapshot = try_load_snapshot("snapshot.pt")
    setup_distributed()
    train(snapshot)

if __name__ == "__main__":
    main()
```

You then launch with something like:

```
torchrun --nnodes=4 --nproc_per_node=1 --rdzv_backend=c10d \
        --rdzv_endpoint=$HEAD_NODE:29500 train_script.py
```

Difficulty:

- Using it: moderate — you must adopt the snapshot style and manage consistent checkpoints.
- Reimplementing: medium-high — particularly the rendezvous and elastic respawn logic.

For Compute4Me, `torchrun` could be the **low-level training engine inside containers**, especially if you want per-job elastic membership of volunteer nodes without writing your own elastic engine.

4.4 Resource Allocation and Workload Scheduling at Scale

Elastic training is one piece. Another is which jobs get which GPUs when, especially in a large data center or cluster.

A recent survey gives a comprehensive overview:

- *"Resource Allocation and Workload Scheduling for Large-Scale Distributed Deep Learning: A Survey"* (Liang et al., 2024)
arXiv: <https://arxiv.org/abs/2406.08115>

4.4.1 What This Survey Covers

It reviews strategies from 2019–2024 for:

- Resource types: GPUs, CPUs, memory, network bandwidth.
- Scheduling granularities:
 - Job-level (which node runs which job).
 - Iteration-level (e.g., scheduling mini-batches across GPUs).
 - Pipeline-level (LLM pipeline parallelism).
- Performance goals:
 - Throughput (jobs/hour).
 - Time-to-accuracy for individual jobs.
 - Fairness or SLA compliance.

It highlights key challenges:

- Heterogeneity in hardware (different GPU types, different network links).
- Mixed workloads (training vs inference vs HPO sharing the same cluster).
- Fault tolerance and preemption.

There is also a companion survey focused on communication efficiency in large-scale DL:

- *"Communication-Efficient Large-Scale Distributed Deep Learning: A Comprehensive Survey"* (Liang et al., 2024)
arXiv: <https://arxiv.org/abs/2404.06114>

These surveys catalog many specialized schedulers (e.g., E-LAS, GPU topology-aware schedulers), but they all assume:

- A managed data center or cloud cluster, not volunteer nodes.
- Trusted and relatively homogeneous resources.

4.4.2 Difficulty to Use vs Reimplement

- **Using insights:** Easy — you can adopt scheduling heuristics (e.g., assign short jobs to faster GPUs, co-locate communication-heavy jobs on nodes with better connectivity).
- **Reimplementing individual schedulers from papers:** Medium-hard — each paper has its own algorithm; integrating them into a working cluster is a non-trivial engineering project.
- **Recreating a full data-center scheduler:** Very hard — think job queueing, multi-tenant fairness, preemption, etc.

For Compute4Me, you can cherry-pick ideas, especially around:

- Heterogeneity-aware scheduling.
- Joint optimization of training and inference workloads.
- Handling resource churn.

4.5 What Is “Solved” vs Still Hard Around Elastic & Heterogeneous Training

4.5.1 Mostly Solved / Common Knowledge

Within the *fixed-world* or cloud-autoscaled cluster setting, the following are relatively mature:

- **Elastic training with controlled membership:**
 - Horovod Elastic can handle node failures and dynamic worker counts when used with autoscaling.^[1]
 - PyTorch torchrun provides fault tolerance and elastic restarts.
- **Cluster autoscaling in the cloud:**
 - Ray autoscaler can grow/shrink clusters across AWS/GCP/Azure.
 - Ray + Horovod integration demonstrates elastic DL on managed clusters.
- **Basic heterogeneity support:**
 - Many schedulers can deal with mixed GPU types by annotating jobs with resource requirements.

These are usable today if you have a cloud account and control over all nodes.

4.5.2 Still Hard / Researchy

The hard problems start when you combine elasticity with **unreliable, heterogeneous, possibly untrusted nodes** — exactly the Compute4Me scenario:

1. Frequent membership churn on consumer internet:

- Most elastic frameworks assume a relatively slow rate of membership change (e.g., VMs joining/leaving occasionally), not frequent volunteer churn.

2. Strong accuracy guarantees under aggressive elasticity:

- EasyScale addresses consistency for certain models and setups, but there's no general recipe for all architectures.

3. Deep heterogeneity:

- Mixed GPUs (8 GB, 12 GB, 24 GB) with unknown network bandwidth and latency.
- Many schedulers assume data-center network regularity (e.g., fat-tree with known topology), not the randomness of internet peers.

4. Security aspects on top of elasticity:

- Elastic training assumes trusted nodes. Adding malicious volunteers who can inject bad gradients into a constantly changing worker pool is still an open problem.

These are exactly the spots where Compute4Me can do something new.

4.6 Compute4Me Design Implications from Section 4

Based on EasyScale, elastic Horovod, torchrun, and the scheduling surveys, here is how they inform a Compute4Me design:

4.6.1 Use Existing Elastic Engines Inside Containers

Rather than inventing your own elastic DDP, you can:

- Use PyTorch `torchrun elastic` or Horovod Elastic *inside each worker container*.
- Treat each container group participating in a training job as a mini elastic cluster.

Your compute fabric (Compute4Me master) then:

- Decides which volunteer nodes are currently part of that job's elastic group.
- Ensures they share a rendezvous configuration (for torchrun) or Horovod settings.

4.6.2 Design Capability-Aware Elastic Scheduling at the Master Level

Combine elasticity with heterogeneity by:

- Profiling nodes (VRAM, FLOPS, bandwidth, reliability) when they join the fabric.
- Building an elastic scheduler that:
 - Adds powerful, stable nodes early to critical jobs.
 - Opportunistically adds short-lived or weaker nodes for extra parallelism when possible.
 - Removes flaky nodes with minimal disruption.
- Borrow EasyScale's concept of **decoupling training logic** from resource allocation:
 - Training algorithm thinks in terms of logical workers and global batch size.
 - Scheduler maps logical workers onto physical nodes dynamically.

4.6.3 Snapshot-First Mentality

Adapt torchrun's snapshot idea:

- Treat all training jobs as first-class **state machines**.
- Regularly snapshot everything that matters (model, optimizer, epoch, RNG state, scheduler state).
- On any failure or membership change:
 - Tear down the job's current process group.
 - Rebuild it with the new set of nodes.
 - Resume from the latest snapshot.

This keeps elasticity and fault tolerance manageable.

4.6.4 Opportunistic Use of Idle GPUs

EasyScale showed that opportunistically harnessing idle GPUs can improve utilization by >60%. For Compute4Me:

- Maintain a **global queue** of DL jobs (train, HPO, inference).
- When a new volunteer node appears:
 - Decide whether to attach it to an elastic training job (e.g., exoplanet model).
 - Or to assign it standalone trials from an HPO search.

In both cases, you leverage elasticity and heterogeneity-aware scheduling together.

4.7 Summary

Section 4's core lesson is: **elastic training is the key enabler for exploiting dynamic, heterogeneous pools of GPUs**, and there are strong existing building blocks (EasyScale ideas, Horovod Elastic, torchrun, Ray autoscaler). What's missing is a system that:

- Uses those building blocks inside containers,
- Applies them to a **volunteer-style, internet-scale cluster** of nodes,
- And layers a **DL-aware, capability-sensitive scheduler** on top.

That system is exactly where Compute4Me can be novel: an elastic training fabric for real-world, unreliable, heterogeneous hardware owned by different users, built on top of proven elastic primitives instead of re-implementing them.

Next section (5) in the larger series will dive into decentralized and volunteer-based distributed deep learning (Hivemind, SWARM, secure distributed training) and how they complement elastic and heterogeneous training in a Compute4Me-style architecture.

